

DCC Overview

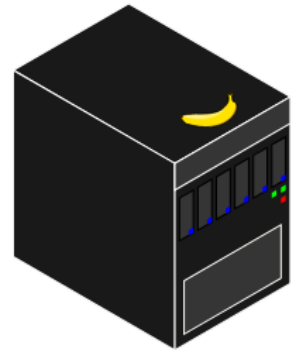
Kayla Wilhoit

Data Lunch

19 Feb 2026

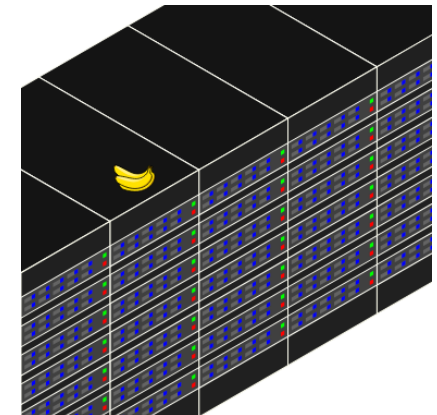
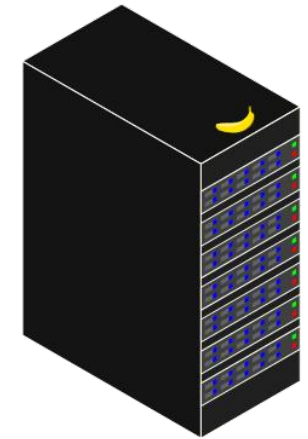
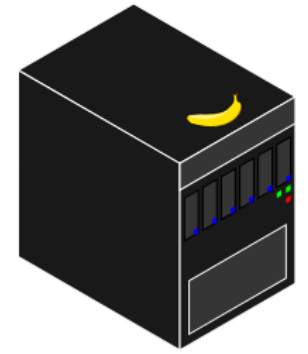
What is the DCC, actually?

- The DCC is essentially a collection of individual computers
- Each computer is called a **node** and has its own CPUs, memory, and storage
 - The DCC has 1250 nodes and 36,000 total CPU cores!



What is the DCC, actually?

- The DCC is essentially a collection of individual computers
- Each computer is called a **node** and has its own CPUs, memory, and storage
 - The DCC has 1250 nodes and 36,000 total CPU cores!
- Nodes are grouped together in **partitions**
 - Partitions help designate nodes for the best suited roles
 - Each partition has a separate **job queue**
 - Many partitions are owned by specific labs, while some are general-use
 - Pick the right partition that suits your job needs!
- When you submit a job to SLURM, you specify which partitions it should send your job to
 - Most likely you will be using a general-use partition (common, scavenger)



General-Use Partitions

- **common** for jobs that will run on the DCC core node
 - 64 nodes, 5804 CPUs
- **scavenger** for jobs that will run on lab-owned nodes in “low priority” (kill and requeue preemption)
 - Includes many but not all lab or department nodes
 - 103 nodes, 9020 CPUs
- **interactive** for debugging and testing scripts. This should be limited to short interactive sessions or short-duration tests and is not intended for use in general analyses. Default resources are low. See below for partition limits per user.
 - 60 nodes, 5280 CPUs

Lab / Department Partition Examples

- **biostat** - Biostatistics and Bioinformatics department partition
 - 7 nodes, 804 CPUs
- **schmidlab** - Amy Schmid’s lab partition
 - 1 node (dcc-schmidlab-02), 76 CPUs

DCC Filesystems

- **Nodes** and **partitions** determine what CPUs run your commands
- The **filesystem** determines where your files live

/hpc/home/(NetID)

- 25 GB (fixed) home directory on the Isilon X-series filers

Where you keep your personal config files and conda environments



/hpc/group/(group name)

- Primary storage on the Isilon X-series filers
- 1 TB group quota (typical)
- One week of daily backups in /hpc/group/(group name)/.snapshot

Where you keep your lab scripts, data, and important things for your lab



/hpc/dctrl/(NetID)

- 500 GB storage allocation for doctoral students

Where you keep your personal scripts and data (especially if your group storage is full)



/work (VAST), /cwork (Ceph): 650 TB each

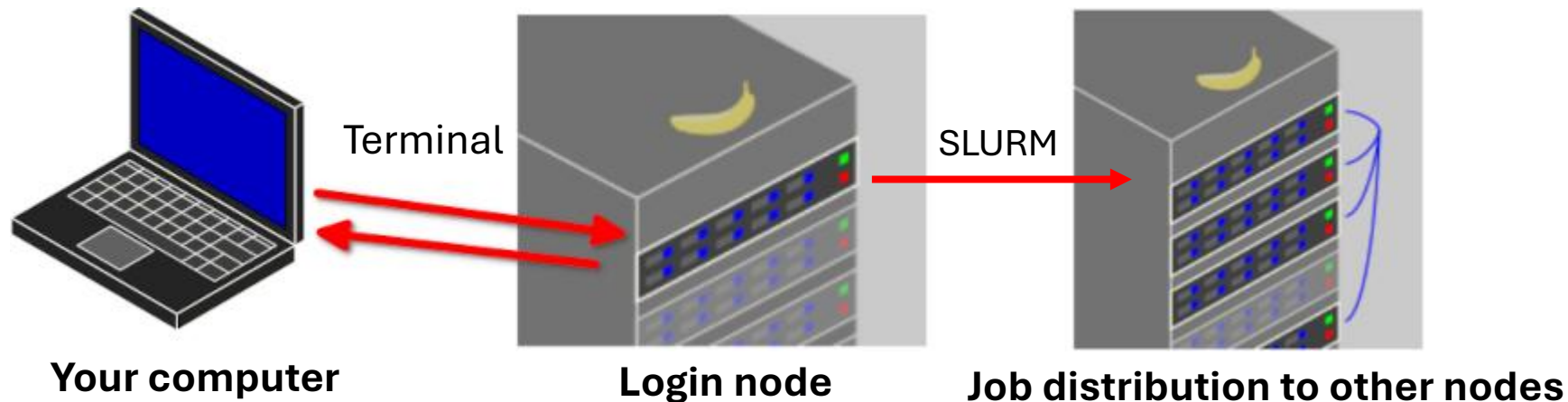
- Temporary storage for large data files
- Place for temporary files associated with running jobs (I/O)
- Not backed-up
- Purged based on file age, files older than 75 days are deleted

Where you can temporarily store huge amounts of data and intermediate files, or test run analyses before moving them somewhere safer



How do you use the DCC?

- When you connect to the DCC, you are placed on a **login node** within your **home directory**
 - Type “hostname” and “pwd” to confirm
 - These are small computers where you can send your jobs to the rest of the DCC using **SLURM**
- Don't run tasks directly on a login node!
 - Rule of thumb – the only commands you should run on the login node are either ls/cd/pwd to look at your folders or start with **sbatch** / **srun** to send your job to a more powerful node



- There are two main ways to work on the DCC: **Interactive jobs** and **batch jobs**

Interactive jobs

- From a login node, run **srun -pty bash -i**
 - This reserves a compute node for you and launches it in the terminal when ready
 - Similar to a Virtual Machine – whatever you type on the command line is run on the compute node with the resources it has
- Best for running multiple short, relatively lightweight jobs
 - Good for when you will be actively debugging things or working with outputs
 - Example: Generating different R plots from data stored in your group storage
 - Example: Testing a big script on a small test dataset for debugging
- Not recommended for long, intensive jobs
 - Don't run “set it and forget it” scripts - you may get kicked off the node eventually
 - Don't run memory-intensive jobs – can fill up the node's temp storage
- Specify memory you need for a session using **--mem=**

```
$ srun --mem=4G --pty bash -i
```

Batch jobs

- Submits your job to the **SLURM** job scheduler
- SLURM does a lot of things:
 - Keeps track of which nodes are free and which are busy
 - Accepts job requests from users
 - Assigns the nodes and resources that fit those jobs
 - Keeps things fair by maintaining a queue
 - Cleans up the nodes afterward to free up resources for new jobs
- You fill out a form using **sbatch** that tells SLURM:
 - How many CPUs you need (-c, --cpus-per-task)
 - How much memory you need (--mem)
 - What partition you want to run on (-p, --partition)
 - To email you when your job is done (--mail)
 - Then add the script or command you want to run!

Batch jobs contd.

- You can use **sbatch** to submit a **one-liner command** or a **formatted shell script**
- One-liner command (not recommended for reproducibility):
 - **sbatch {parameters} --wrap="{command}"**

```
sbatch --ntasks=1 --mem=1G --wrap="seqkit translate VNII_chr1.fa > VNII_chr1_protein.fa"
```

- Formatted shell script:
 - Use #SBATCH to specify each parameter on a line
 - Specify any environments or directory changes
 - Specify stdout or error files
 - Run in terminal:
 - **sbatch locate.sh**

```
#!/bin/bash

#SBATCH --mem=2GB
#SBATCH --mail-type=BEGIN
#SBATCH --mail-type=END
#SBATCH --mail-type=FAIL
#SBATCH -p common,scavenger
#SBATCH -c 4
#SBATCH --output=/work/ktw31/VNII_to_cockatoo/seqkit_locate.out

source /hpc/home/ktw31/yes/etc/profile.d/conda.sh # where to find my conda environments
conda activate genomics_env # activate the environment with the correct software (seqkit)

cd /work/ktw31/VNII_to_cockatoo # change to directory with the file to use

# Locate the minisatellite M13 sequence in the reference genome, ignoring case, using 4 threads,
seqkit locate -i -m 3 -j 4 -p GAGGGTGGCGGTTCT VNII.ref.fa > m13_hits.tsv
```

Checking on your jobs

- When you submit your job, you will get a job ID

```
(base) ktw31@dcc-login-04 /work/ktw31/VNII_to_cockatoo $ sbatch locate_m13_script.sh
Submitted batch job 43534655
```

- You can check the status of that job by using **scontrol show job {jobid}**

```
(base) ktw31@dcc-login-04 /work/ktw31/VNII_to_cockatoo $ scontrol show job 43534655
JobId=43534655 JobName=locate_m13_script.sh
UserId=ktw31(1525280) GroupId=dukeusers(1000000) MCS_label=N/A
Priority=11809 Nice=0 Account=wraylab QOS=normal
JobState=COMPLETED Reason=None Dependency=(null)
```

- Use **scancel {jobid}** to cancel an existing job
- Check on all of your jobs using **squeue -u {netid}** or **squeue --me**
 - Add the **-l** tag for full column names and easier reading

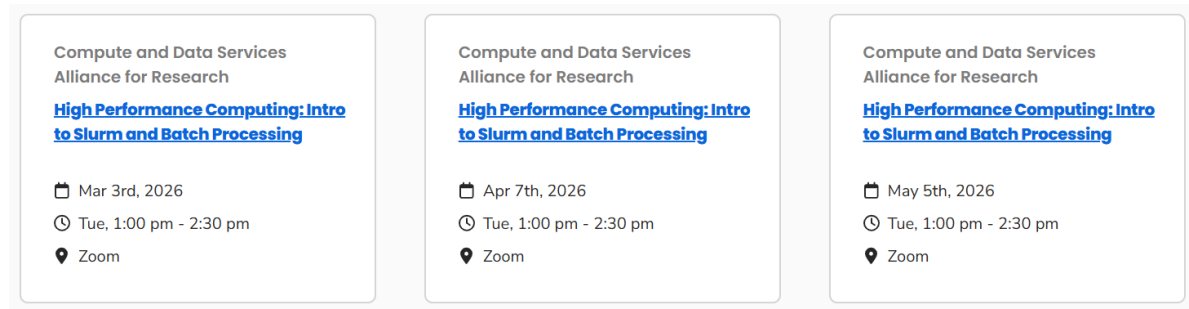
```
ktw31@dcc-login-04 /work/ktw31/VNII_to_cockatoo $ squeue -l -u ktw31
18 16:52:14 2026
JOBID PARTITION NAME USER STATE TIME TIME_LIMI NODES NODELIST(REASON)
43527805 common bash ktw31 RUNNING 1:04:35 1-00:00:00 1 dcc-core-19
43534775 common,sc locate_m ktw31 PENDING 0:00 1-00:00:00 1 (Priority)
```

- Use **sacct -u {netid}** to see your job history and memory usage
 - **sacct -j {jobid} --units=G** shows the memory a job used in Gb and time elapsed

User	JobID	State	Partition	Start	Elapsed	NodeList	MaxRSS	ReqMem	VCPU
ktw31	43534740	COMPLETED	common	2026-02-18T16:50:24	00:00:08	dcc-core-10		2G	4
	43534740.batch	COMPLETED		2026-02-18T16:50:24	00:00:08	dcc-core-10	0.67G		4
	43534740.extern	COMPLETED		2026-02-18T16:50:24	00:00:08	dcc-core-10	0		4

Dos and Don'ts

- **Do** attend a DCC training workshop if you are scared about using DCC!



- **Do** reach out to rescomputing@duke.edu with questions if you want to run something specific or want to request new software installs

For the DCC, you can change this to something like

```
#!/bin/bash
#SBATCH -e slurm.err
snakemake --cluster "sbatch -p common -N 1" --jobs NUM_JOBS_TO_SUBMIT
```

- **Don't** make huge temporary files on interactive nodes or you may crash a node and get a disappointed email from ResComputing :(

[9:31 AM] Mike Newton
ktw31 filled up / on dcc-core-19

Tom, ktw31 from wraylab filled up / on dcc-core-19 by writing a directory in /tmp (ee0948d8e8f850fcb4447656f754b35a) that has 60G. I've removed the directory and put the node back into service.

Useful links:

- DCC User Guide:
 - <https://oit-rc.pages.oit.duke.edu/rcsupportdocs/dcc/>
- Upcoming DCC training workshops:
 - <https://oit-rc.pages.oit.duke.edu/rcsupportdocs/help/training/>
- Most recent DCC intro slides from workshop:
 - <https://duke.box.com/s/zosu3w34iap7o4yuvlq44zep8ic27byq>
- Intro to HPC – Software carpentries lesson
 - <https://carpentries-incubator.github.io/hpc-intro/10-hpc-intro.html>
- Claim your PhD Student Allocation:
 - <https://oit-rc.pages.oit.duke.edu/rcsupportdocs/help/Phdstudentaccess/>
- Find your lab's point of contact to add you to the group:
 - <https://rtoolkits.web.duke.edu/projects/report>